Large Language Model Foundations

Generative AI handbook, volume 1 · Version 2026.2

> Sample foundation document written for testing a domain-expert RAG. It summarises widely used concepts; check vendor documentation for current limits and product details.

1. What a large language model is

A large language model (LLM) is a neural network trained to predict the next token in a sequence. Trained on trillions of tokens of text and code, it learns grammar, facts, reasoning patterns and styles well enough that repeated next-token prediction produces useful answers, code and summaries. Everything an LLM does, from translation to tool calling, is next-token prediction conditioned on the text in its context window.

Three stages usually produce a modern assistant model. **Pre-training** learns general language ability from a very large corpus. **Instruction tuning** (supervised fine-tuning, SFT) teaches the model to follow instructions using curated prompt-response pairs. **Preference optimisation**, such as reinforcement learning from human feedback (RLHF) or direct preference optimisation (DPO), aligns responses with human preferences for helpfulness and safety.

2. Tokens and tokenisation

Models read tokens, not characters or words. A tokenizer such as byte-pair encoding (BPE) or SentencePiece splits text into sub-word units from a fixed vocabulary, typically 32,000 to 200,000 entries. In English one token is roughly four characters or three quarters of a word. Scripts that are under-represented in the tokenizer's training data, such as Tamil, often need several times more tokens for the same meaning, which raises cost and uses more of the context window.

Concept	What it means in practice
Vocabulary	The fixed set of token pieces the model knows. Unknown text is broken into smaller pieces, down to bytes.
Context window	The maximum number of tokens (prompt plus output) the model can attend to in one call, for example 32k, 128k or 1M tokens.

Token budget	Pricing and latency scale with input and output tokens. Output tokens are usually slower and more expensive than input tokens.
Special tokens	Markers for roles (system, user, assistant), tool calls and end of turn, defined by the model's chat template.

3. The transformer architecture

Almost all current LLMs are decoder-only transformers. Each layer has two parts: **self-attention**, which lets every token look at every earlier token and decide which ones matter, and a **feed-forward network**, which transforms each token's representation. Stacking dozens of layers lets the model build increasingly abstract representations.

3.1 Attention

For each token the model computes a query, a key and a value vector. Attention weights come from the similarity of a token's query with the keys of earlier tokens; the output is the weighted sum of their values. Multi-head attention runs several of these in parallel so different heads can track different relationships, such as syntax, co-reference or code structure. Because attention compares every token with every other, its cost grows with the square of sequence length, which is why long contexts are expensive.

3.2 Positional information

Attention by itself has no notion of order, so position is injected. Rotary position embeddings (RoPE) are the most common choice today; they encode relative position and can be stretched to longer contexts with techniques such as position interpolation.

3.3 The KV cache

During generation the keys and values of earlier tokens are stored in a key-value (KV) cache so they are not recomputed for every new token. The KV cache often uses more GPU memory than people expect: it grows linearly with context length and with the number of concurrent requests. Grouped-query attention (GQA) shares keys and values across heads to shrink it.

4. Mixture of experts

A mixture-of-experts (MoE) model replaces some feed-forward layers with many parallel "expert" networks and a router that sends each token to a small number of them, often 2 to 8 out of 64 to 256. The model's **total parameters** can be very large while the **active**

parameters per token stay small, so it has the knowledge capacity of a big model with the compute cost of a much smaller one. The catch is memory: all experts must be loaded, so an MoE model needs as much memory as its total parameter count implies, even though each token touches only a fraction of them.

5. Generation and sampling parameters

Parameter	Effect	Typical use
temperature	Scales the probability distribution. Lower is more deterministic; higher is more varied.	0 to 0.3 for extraction, RAG and code; 0.7 to 1.0 for brainstorming.
top_p (nucleus)	Samples only from the smallest set of tokens whose probabilities add up to p.	0.9 to 1.0. Tune temperature or top_p, not both at once.
top_k	Samples only from the k most likely tokens.	20 to 100 in open-weight runtimes.
max_tokens	Upper limit on output length.	Set it to protect cost and latency; reasoning models need room for thinking tokens.
stop sequences	Strings that end generation.	Structured outputs and delimiters.
repetition / frequency penalty	Discourages repeating tokens.	Small values for long-form text.
seed	Makes sampling repeatable where supported.	Tests and evaluations.

Temperature versus top-p. Temperature changes how peaked the whole distribution is; top-p cuts off the long tail after the distribution is computed. With temperature 0 the model always takes the most likely token (greedy decoding), and top-p has no effect.

6. Reasoning models

Reasoning models are trained to produce an internal chain of thought before the final answer. They do better on maths, planning and multi-step code changes but spend more

output tokens and time. Many APIs expose a reasoning effort setting (low, medium, high) that trades quality for latency and cost. For retrieval-grounded question answering, low effort is usually enough; for complex analysis, higher effort pays off.

7. Memory and quantisation

Model weights need memory equal to the number of parameters times the bytes per parameter. A 70-billion-parameter model needs about 140 GB in 16-bit precision, 70 GB at 8 bits and about 35 to 40 GB at 4 bits, before counting the KV cache.

Format	Bytes per parameter	Notes
FP32	4	Training master weights; rarely used for inference.
BF16 / FP16	2	Standard inference precision on GPUs.
FP8 / INT8	1	Near-lossless on modern GPUs.
4-bit (GPTQ, AWQ, GGUF Q4)	about 0.5	Good quality for most tasks; the usual choice for local deployment.
2 to 3-bit (GGUF Q2/Q3)	0.25 to 0.4	Fits very large models on one GPU; quality loss becomes noticeable, so evaluate on your own tasks.

GGUF is the file format used by llama.cpp. It stores quantised weights with metadata and supports mixed precision, where sensitive layers keep more bits than others. Dynamic quantisations choose per-layer precision to keep quality at low average bit widths.

8. Hallucination

A hallucination is fluent output that is not supported by facts or by the provided context. It happens because the model is optimised to produce plausible text, not to know when it does not know. Mitigations, from most to least effective: ground answers in retrieved sources and require citations; allow and reward "I don't know"; lower temperature; ask for quotes before conclusions; verify outputs with a second pass or deterministic checks; and evaluate on questions whose answers are known.

9. Multimodal models

Vision-language models accept images alongside text. An image encoder turns the picture into tokens that the language model attends to. They can transcribe scanned documents, read tables and charts, describe diagrams and answer questions about screenshots. Accuracy drops on low-resolution scans, handwriting, dense tables and scripts that were rare in training, so production pipelines often combine a vision model for understanding with OCR for exact character positions.

10. Choosing a model

Need	Prefer
Lowest latency and cost at scale	Small or MoE models, quantised, possibly fine-tuned for the task.
Hard reasoning, long agent tasks	Frontier or reasoning models with higher effort settings.
Data must stay on premises	Open-weight models served with vLLM, llama.cpp or TGI.
Many languages	Models with multilingual tokenizers and evaluations in your target languages.
Structured output	Models and servers that support JSON schema or constrained decoding.